A Campus-Scale Radio Interferometer in Software

Simulation, real-time FX correlation, imaging and array design

Alireza Poorhasan Astaneh
Department of Physics, Sharif University of Technology — supervised by Dr. Reza Rezaee

Original project 2024 · extended edition October 24, 2026

Abstract

The bachelor project behind this report supported a group building radio receivers to be placed at different locations on the Sharif University campus. The task was to handle the receivers' data streams and compute the phase differences between them, a workload that grows as $N(N-1)/2$ with the number of receivers N. This report describes an extended, self-contained version of that work: `radiotel`, an end-to-end software interferometer. It simulates the voltage streams that receivers would record from a sky of point sources. It correlates them with an FX correlator that has interchangeable NumPy, PyTorch and hand-written C (AVX2, OpenMP) back ends, and runs the C kernel inside a multi-threaded streaming daemon fed over TCP. The visibilities are calibrated with StEFCal, the sky is imaged by Earth-rotation synthesis and Högbom CLEAN, and the receiver layout is optimised within the campus footprint. On a laptop CPU (Intel Core i5-10500H CPU @ 2.50GHz) the AVX2+OpenMP kernel correlates 64 receivers 1.79× faster than real time at 2.4 MS/s, and a cloud GPU reaches 9.1×. The streaming daemon sustains 12.40 MS/s per receiver for 8 receivers, including networking. Simulated observations are imaged with fluxes recovered to within about 5 %.

Contents

1 Background and motivation 1

2 Signal model 2

3 Phase-difference estimation 3

4 The FX correlator 3

5 Real-time streaming correlator 5

6 Imaging and calibration 6

7 Where to put the receivers 7

8 Limitations and future work 7

9 Reproducing the results 8

1 Background and motivation

A radio interferometer replaces one very large dish with several small receivers whose signals are combined. For a distant source at direction $\hat{\mathbf{s}}$, two receivers separated by the baseline $\mathbf{b}$ see the same wavefront with a geometric delay $\tau_g = \mathbf{b} \cdot \hat{\mathbf{s}}/c$. At frequency ν this delay is a phase difference

$2\pi\nu\tau_g$, which locates the source with a resolution of roughly $\lambda/|\mathbf{b}|$. A 300 m baseline at the 21 cm hydrogen line resolves about 2.4 arcmin, compared with about 6° for a 2 m dish.

The original project assumed a simple picture: two receivers, a sinusoidal signal, and an estimate of the phase difference. It simulated TCP data streams in C, compared FFT, Hilbert-transform and cross-correlation estimators, and experimented with AVX intrinsics and PyTorch on a GPU. A notebook also computed baseline tracks under Earth rotation and searched for good receiver layouts. The prototypes are preserved unchanged in `legacy/`, together with a list of their known defects.

This edition keeps the original goals and develops them along four directions:

1. **Realistic signals.** Astronomical sources emit noise rather than tones, so the phase difference must be measured per frequency channel through cross-correlation. The single-tone estimators are benchmarked against the Cramér–Rao bound to motivate this step (Section 3).

2. **Scalable correlation.** The $N(N-1)/2$ pairwise computation is implemented as an FX correlator with several back ends and benchmarked up to $N = 64$ (Section 4).

3. **Real-time data handling.** The streaming prototype is rebuilt as a correct, measurable daemon (Section 5).

4. **Science output.** Correlated data are turned into calibrated sky images, and the result is used to choose receiver positions (Sections 6–7).

2 Signal model

Receiver positions are local east–north–up (ENU) vectors on the campus. A baseline (i, j) with $i < j$ is $\mathbf{b}_{ij} = \mathbf{r}_j - \mathbf{r}_i$. In the frame that tracks the phase centre at hour angle H and declination δ, the baseline coordinates in wavelengths are

$$\begin{pmatrix} u \\ v \\ w \end{pmatrix} = \frac{1}{\lambda} \begin{pmatrix} \sin H & \cos H & 0 \\ -\sin\delta\cos H & \sin\delta\sin H & \cos\delta \\ \cos\delta\cos H & -\cos\delta\sin H & \sin\delta \end{pmatrix} \mathbf{b}_{XYZ}, \tag{1}$$

where $\mathbf{b}_{XYZ}$ is the baseline rotated from ENU to the equatorial frame at latitude $\phi = 35.70°$. As the Earth rotates, each baseline traces an ellipse in the (u, v) plane.

Each source k with direction cosines (l_k, m_k) and flux S_k is modelled as circular complex Gaussian noise $s_k(t)$. After delay tracking, receiver a records

$$x_a(t) = g_a\Big[\sum_k s_k(t-\tau_{ak})\, e^{-2\pi i\nu_0\tau_{ak}} + n_a(t)\Big], \qquad \tau_{ak} = -\frac{u_a l_k + v_a m_k + w_a(n_k-1)}{c}, \tag{2}$$

with independent receiver noise n_a and an optional complex gain g_a. The delay is applied exactly, as a phase ramp in the frequency domain, so fractional-sample delays are represented correctly. The correlation of two receivers then obeys the van Cittert–Zernike relation

$$V_{ij}(\nu) = \langle X_i(\nu) X_j^*(\nu)\rangle = g_i g_j^* \sum_k S_k\, e^{-2\pi i(u l_k + v m_k + w(n_k-1))}. \tag{3}$$

Default parameters are $\nu_0 = 1420.4$ MHz, a complex-baseband bandwidth of 2.4 MHz (typical of software-defined radios), a phase centre at $\delta = 40°$ observed for 8 hours, and 64 channels. These are assumptions for the simulation, not specifications of the actual receivers.

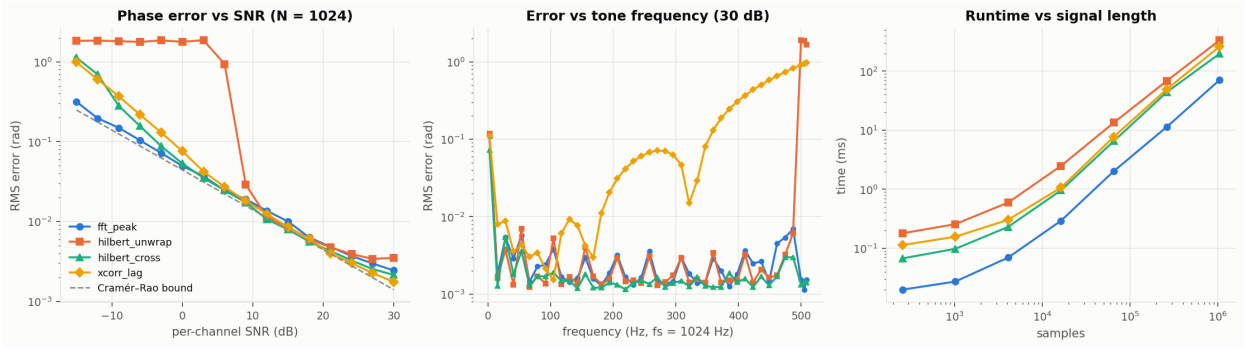


Figure 1: Two-tone phase estimators. Left: RMS error against SNR, with the Cramér–Rao bound (dashed). Middle: error against tone frequency at 30 dB. All methods degrade near DC. The lag method degrades steadily towards Nyquist because parabolic peak interpolation becomes biased with fewer samples per period, and Hilbert unwrapping fails at Nyquist. Right: runtime against signal length.

3 Phase-difference estimation

The original comparison is repeated with Monte-Carlo trials, phase-wrapped errors and an off-bin tone (97.3 Hz at $f_s = 1024$ Hz, $N = 1024$). Figure 1 compares four estimators: the cross-spectrum at the FFT peak bin, the legacy Hilbert method (mean of the unwrapped instantaneous-phase difference), the averaged analytic cross-product $\arg \sum a_2 a_1^*$, and the cross-correlation lag (with the legacy whole-sample quantisation removed). For a tone in white noise, any unbiased estimator obeys

$$\operatorname{var}(\hat{\varphi}_2 - \hat{\varphi}_1) \geq \frac{2}{N \cdot \mathrm{SNR}}, \qquad \mathrm{SNR} = \frac{A^2}{2\sigma^2}. \tag{4}$$

The cross-product estimators follow the bound down to low SNR. At 0 dB the analytic cross-product reaches 0.053 rad, against a bound of 0.044 rad. The legacy unwrapping method breaks down: a single 2π slip biases the whole mean, and its error at 0 dB is 1.78 rad.

Why correlation is the robust choice. The FFT peak bin looks best here because the test is its ideal case: a pure, steady tone puts all of its energy into one bin. That advantage disappears as soon as the signal departs from this ideal. In a supplementary test with a drifting oscillator (90 Hz to 110 Hz over the record, 6 dB SNR), the original project's FFT method gave an RMS error of 0.58 rad. That method reads each signal's phase at its own spectral peak and subtracts the two. Time-domain correlation gave 0.024 rad, about 25× better, because the drifting energy spreads over many bins while correlation still uses all of it. This agrees with standard practice in signal processing: practitioners rely on cross-correlation of the two signals, not on phases read off a spectral peak. The FFT remains essential, but as the fast way to compute that correlation, not as a peak detector.

The best-performing estimators above all have this form. The "FFT peak" estimator is itself a correlation, because $\arg X_2 X_1^*$ is the cross-power spectrum, the Fourier transform of the cross-correlation. A real sky signal is broadband noise rather than a tone, so the estimator that generalises is this cross-product evaluated in every frequency channel and averaged over time. That is exactly the FX correlator.

4 The FX correlator

The F-step splits each stream into blocks of n_{chan} samples and applies a unitary FFT to each. The X-step then averages $X_i X_j^*$ over blocks for every baseline and channel. For S samples per receiver the X-step performs $\frac{1}{2} N(N-1)S$ complex multiply–accumulates (8 real flops each) and dominates for $N \gtrsim 8$. Every back end exposes the same call, `correlate(volts, nchan, backend)`:

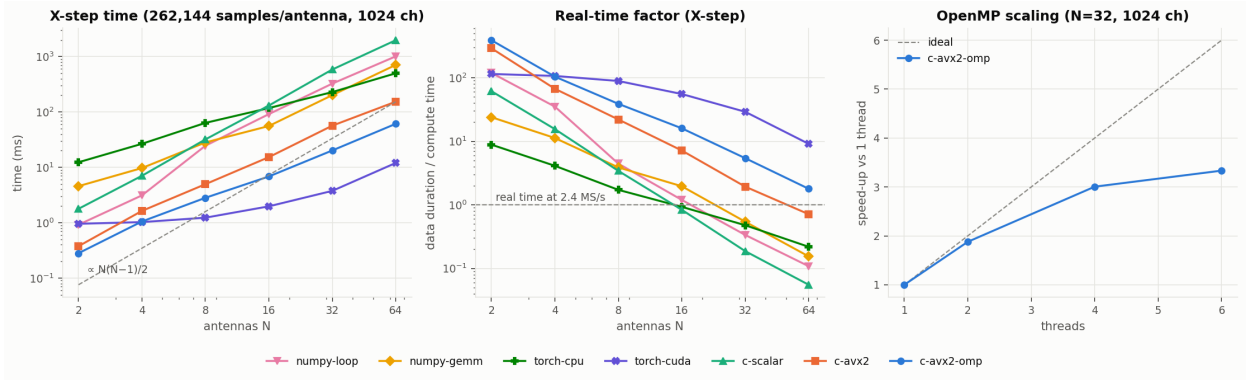


Figure 2: X-step time (left) and real-time factor (middle) against the number of receivers, at 1024 channels and 262,144 samples per receiver. Right: OpenMP thread scaling at $N = 32$.

Table 1: X-step real-time factor at 2.4 MS/s per receiver, 1024 channels (higher is better; > 1 keeps up with a live stream). `torch-cuda` measured on a cloud GPU, other rows on the CPU host; –: skipped (estimated >20 s); pending: no GPU results available.

back end	$N = 2$	$N = 4$	$N = 8$	$N = 16$	$N = 32$	$N = 64$
`numpy-loop`	119	35	4.45	1.2	0.336	0.108
`numpy-gemm`	23.9	11.2	3.88	1.96	0.542	0.154
`torch-cpu`	8.84	4.11	1.72	0.929	0.477	0.219
`torch-cuda`	114	106	88.7	55.3	29.1	9.1
`c-scalar`	61.3	15.5	3.42	0.836	0.187	0.0553
`c-avx2`	290	66.8	22	7.15	1.93	0.712
`c-avx2-omp`	388	104	38.8	16	5.41	1.79

back end	X-step implementation
`numpy-loop`	one baseline at a time, vectorised over channels
`numpy-gemm`	per channel $R = SS^H$ (batched BLAS `cgemm`)
`torch-cpu/cuda`	the same GEMM formulation in PyTorch
`c-scalar`	plain C, auto-vectorisation disabled (reference)
`c-avx2`	AVX2+FMA intrinsics, 8 channels per iteration
`c-avx2-omp`	as above, OpenMP across baselines

The AVX2 kernel avoids a shuffle per complex product. A `__m256` register holds four interleaved complex values, and the kernel accumulates $P \mathrel{+}= a \odot b$ and $Q \mathrel{+}= a \odot \mathrm{swap}(b)$ with fused multiply–adds over all blocks. Only at the end does it form $\mathrm{Re} = P_{\mathrm{even}} + P_{\mathrm{odd}}$ and $\mathrm{Im} = Q_{\mathrm{odd}} - Q_{\mathrm{even}}$, the real and imaginary parts of $a\,\bar{b}$.

The memory layout turned out to matter more than the arithmetic. A first version walked the spectra baseline by baseline. It re-read each antenna $N - 1$ times from DRAM with a stride of $8\,n_{\mathrm{chan}}$ bytes, reached only about 5 GFLOP/s, and did not speed up with more threads because every core competed for the same memory bandwidth. The final kernel is cache-blocked. It copies an 8-channel block of all antennas into a contiguous buffer (one 64-byte line per antenna and time step), forms every baseline from it while it sits in L1/L2, and register-tiles two baselines per load. OpenMP then parallelises over channel blocks. This raised single-core throughput about fourfold and made the kernel scale with cores. All back ends agree with the reference to float32 precision (tests in `tests/test_correlator.py`).

Figure 2 and Table 1 summarise the results. The cost follows the expected $N(N-1)/2$ curve. Explicit SIMD gives a 8.6× gain over scalar C at $N = 16$, and OpenMP adds a further 3.3× on 6 threads, peaking at 69 GFLOP/s. At $N = 64$ (2016 baselines) the AVX2+OpenMP kernel is 11.6× faster than the NumPy GEMM formulation. That formulation is limited by the small, skinny matrices involved ($N \times n_{\mathrm{spec}}$ per channel), which suit BLAS poorly on a CPU but suit GPUs well.

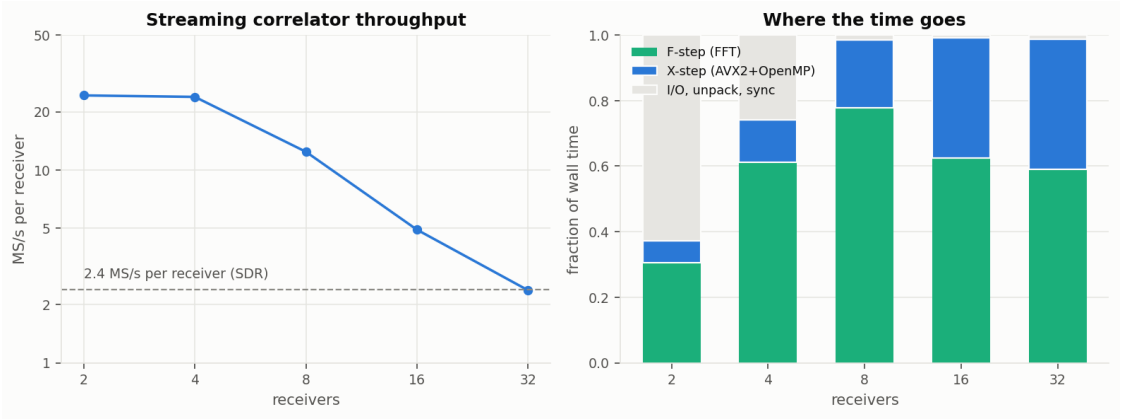


Figure 3: Streaming daemon on one machine (localhost). Left: sustained throughput per receiver, including sockets, unpacking, FFT and X-step. Right: share of wall time per stage.

The same GEMM formulation in PyTorch was run on a cloud GPU (Tesla T4, via `benchmarks/gpu_colab.ipyn` Each call has a floor of about 1 ms from kernel launches and the baseline gather, so for two to four receivers the CPU kernel is as fast or faster. From $N = 8$ the GPU pulls ahead. At $N = 64$ it reaches 352 GFLOP/s and 9.1× real time, 5.1× faster than the AVX2+OpenMP kernel. These timings exclude host–device transfers. In a deployment the spectra would be produced on the GPU (F-step there too), so only the visibilities would need to come back.

5 Real-time streaming correlator

The original C prototype streamed two sine waves over TCP. Its receiving threads wrote into shared buffers without holding the lock, never matched blocks between receivers, and rebuilt FFTW plans for every block. The replacement has two programs:

- `rx_sim` simulates N receivers, each a thread with its own TCP connection. Each sends 32-byte headers (antenna id, sequence number, timestamp) followed by 4096 samples of 8-bit I/Q, as an SDR would. All receivers start their sample clocks at a common epoch, as GPS-locked receivers would. Senders either run on a fixed schedule or go as fast as TCP back-pressure allows. On a fixed schedule, a frame that back-pressure delays beyond a 32-frame (55 ms) receiver buffer is counted as an *overrun*, i.e. data a real receiver would lose.

- `corrd` runs one receiving thread per connection. Each thread writes complete frames into a per-receiver ring buffer under a lock and blocks when it is `ring` frames ahead, so no data are dropped silently. A correlator thread waits until frame s has arrived from *every* receiver and unpacks it into the current integration. Every 16 frames it runs the F-step (a pre-planned radix-2 FFT, parallel over receivers) and *one* AVX2 X-step over the whole integration, then writes a visibility dump. Integration is simply a longer sum in the X-step.

Three engineering iterations were needed to make the daemon real-time:

1. Calling the multi-threaded X-step once per 4096-sample frame spent most of the time in OpenMP fork/join, so it reached only 2.2 MS/s per receiver at $N = 8$. Batching a whole integration into one call, with a parallel FFT, fixed that.

2. Each simulated receiver originally started its clock when it connected. Early receivers ran ahead, were throttled by the alignment window, and showed spurious lag. A common start epoch fixed that.

3. With the OpenMP pool on all hardware threads, the 32 socket threads (senders and receivers) were starved during compute bursts. With the pool limited to half the CPUs, 16 receivers went from hundreds of overruns to none.

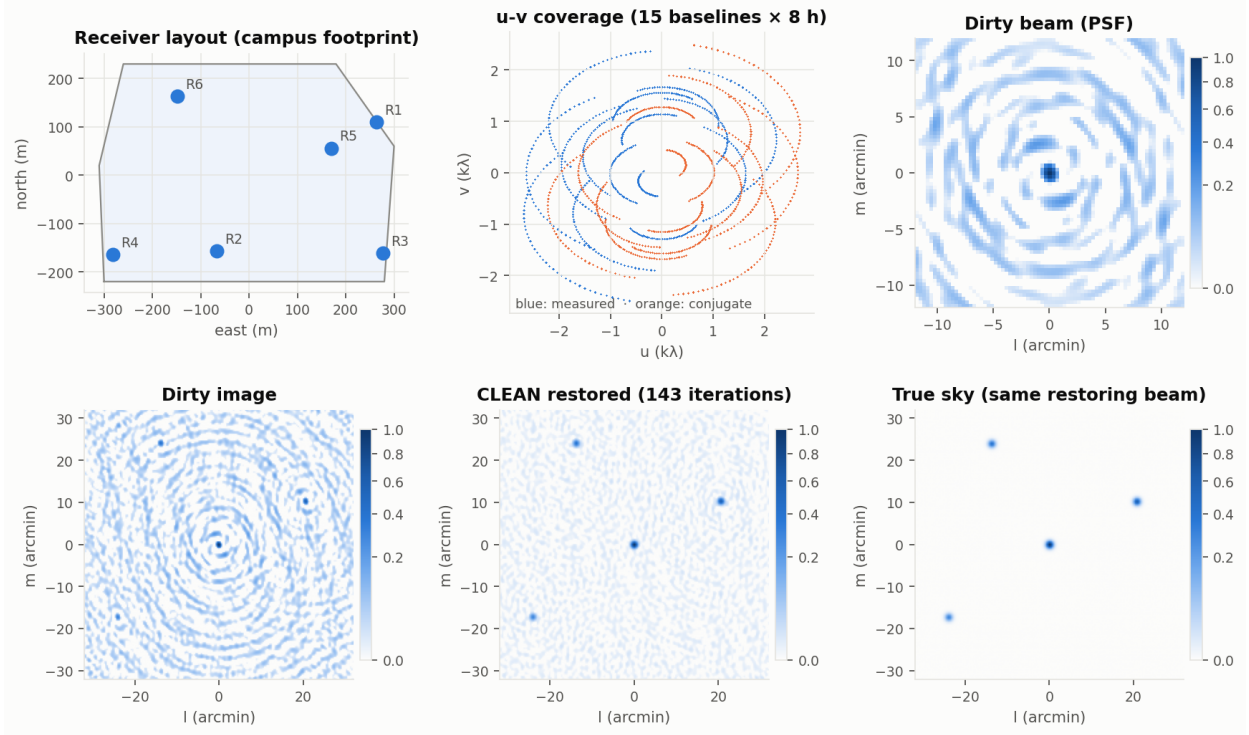


Figure 4: End-to-end simulation: six receivers on the campus footprint, 48 correlator snapshots over 8 hours, gridding, dirty image, and Högbom CLEAN. Images use a square-root stretch.

On one machine, sharing the CPU with the simulated senders, the daemon sustains 24.31, 12.40, 4.89 and 2.38 MS/s per receiver for 2, 8, 16 and 32 receivers (Figure 3). That is 5× real time at $N = 8$, 2× at $N = 16$, and right at the limit for $N = 32$. Paced at 2.4 MS/s, the 8- and 16-receiver runs had 0 and 0 overruns, with maximum lags of 25 and 17 ms. In this regime the simple radix-2 FFT, not the correlation, takes most of the time (right panel), so an optimised FFT library would be the next improvement. The dumps match the offline NumPy correlator on the same quantised samples to a relative difference of about 10^{-6}.

6 Imaging and calibration

Visibilities from all snapshots and channels are gridded onto a fourfold-oversampled (u, v) grid. Oversampling suppresses the phase errors of nearest-neighbour gridding away from the phase centre. The inverse FFT gives the dirty image, the true sky convolved with the dirty beam. Figure 4 shows the full pipeline starting from simulated voltages: the 15 baselines trace arcs in the (u, v) plane, and the resulting dirty beam (FWHM about 1.1 arcmin) has sidelobes of about 0.2. Högbom CLEAN removes them and recovers the four source fluxes (1.0, 0.6, 0.4, 0.3) to within about 5 %. The w-term is neglected, which is accurate for fields within about 0.5° on these baselines. A wider field would need w-projection.

Real receivers add unknown complex gains (cable lengths, amplifiers, local oscillators), so $V_{ij}^{\mathrm{obs}} = g_i g_j^{*} V_{ij}$. With N unknowns and $N(N-1)/2$ measurements, a short observation of a calibrator at the phase centre fixes the gains. StEFCal, an alternating least-squares solver, recovers injected phase errors of about 1 rad to within 0.005 rad (Figure 5). Closure phases summed around baseline triangles cancel all antenna phases exactly. They stay within noise (about 1.5°) of the true sky, while uncorrected baseline phases are off by about 60° on average.

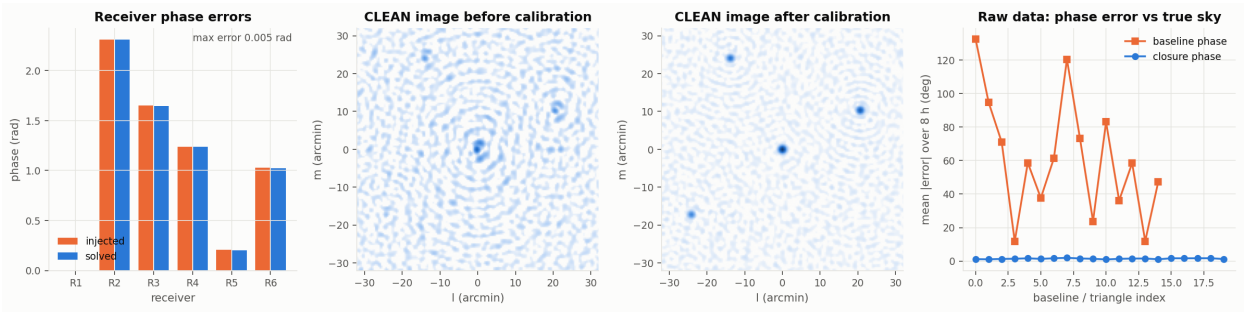


Figure 5: Gain calibration. Receivers are given ∼1 rad phase and 10 % amplitude errors. StEFCal solves them from a short calibrator scan, which restores the image. Closure phases of the raw data match the true sky, whereas baseline phases do not.

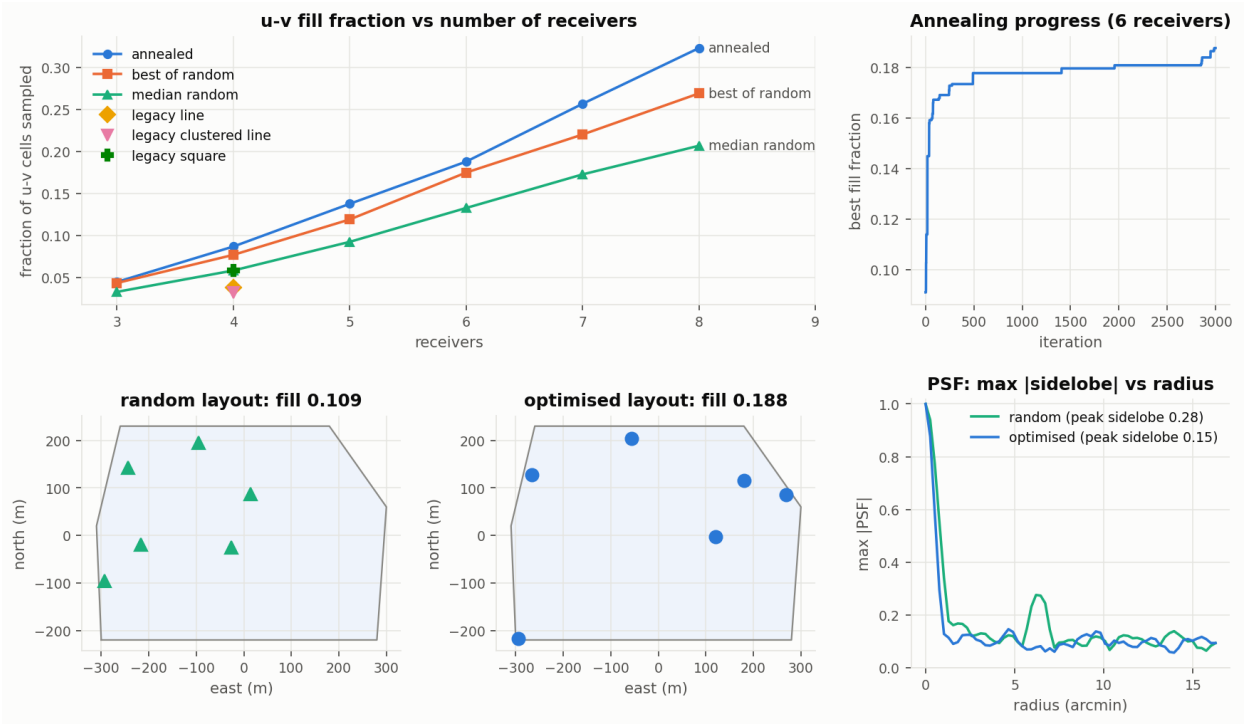


Figure 6: Layout optimisation. Top: (u, v) fill fraction against the number of receivers for the original hand-made layouts, random placements and simulated annealing. Bottom: a random and an optimised 6-receiver layout and the radial sidelobe envelope of their dirty beams.

7 Where to put the receivers

The original notebook scored layouts by plotting their (u, v) tracks and counting dark pixels. Here the same idea is computed directly. The score is the fraction of cells of a 64×64 (u, v) grid, within the disk the campus can reach, that the Earth-rotation tracks visit. Simulated annealing then moves one receiver at a time inside the campus polygon, keeping a minimum separation of 15 m. With four receivers, the original square layout fills 0.058 and the line 0.038, while annealing reaches 0.087. With eight receivers, annealing reaches 0.323, against 0.269 for the best of 200 random layouts and 0.207 for the median (Figure 6). Better coverage translates directly into lower dirty-beam sidelobes, which makes CLEAN's job easier.

8 Limitations and future work

- **GPU timings** exclude host–device copies and come from a shared cloud GPU; the streaming daemon itself is CPU-only. Moving the F- and X-steps of `corrd` to the GPU, ideally with

tensor-core GEMMs, is the natural next step for arrays beyond about 16 receivers.

- **Single-machine streaming.** The receivers and the correlator share one CPU over loopback, and streaming time is dominated by a simple radix-2 FFT; FFTW or a batched SIMD FFT would raise the receiver count further. A real deployment needs distributed receivers, a common clock (GPS-disciplined oscillators or White Rabbit), and coarse delay correction before correlation.
- **Imaging simplifications.** The imager uses nearest-neighbour gridding, no w-term, no primary-beam correction, and point sources only.
- **No real data.** No real data were recorded. The natural next step is to replay recordings from the campus receivers through `rx_sim -input`.
- **Possible extensions.** Polyphase filter banks instead of a plain FFT; RFI flagging; tensor-core correlation; redundant-baseline calibration; HI-line spectral imaging.

9 Reproducing the results

```
make lib       # C library + streaming tools (gcc, OpenMP, AVX2)
make test      # pytest suite
make examples  # figures in figs/
make bench     # benchmarks/results/*.csv
make report    # this document
```

References

[1] A. R. Thompson, J. M. Moran, G. W. Swenson, *Interferometry and Synthesis in Radio Astronomy*, 3rd ed., Springer (2017).

[2] G. B. Taylor, C. L. Carilli, R. A. Perley (eds.), *Synthesis Imaging in Radio Astronomy II*, ASP Conf. Ser. 180 (1999).

[3] J. A. Högbom, Aperture synthesis with a non-regular distribution of interferometer baselines, *A&AS* 15, 417 (1974).

[4] S. Salvini, S. J. Wijnholds, Fast gain calibration in radio astronomy using alternating direction implicit methods, *A&A* 571, A97 (2014).

[5] S. M. Kay, *Fundamentals of Statistical Signal Processing: Estimation Theory*, Prentice Hall (1993).

[6] M. A. Clark, P. C. La Plante, L. J. Greenhill, Accelerating radio astronomy cross-correlation with graphics processing units, *IJHPCA* 27(2), 178 (2013).